LEARN ML BY BUILDING

LECTURE 04

Gradient Descent and Optimization

From objective functions to parameter updates

Training searches for parameter values that reduce a loss. Optimization methods differ in the information they use: function values, gradients, or curvature.

Learning goals. By the end of this lecture, you should be able to

1. formulate a learning task using variables, an objective, and constraints;
2. distinguish local and global minima and explain convexity;
3. derive gradient descent and explain the roles of the learning rate and data per update; and
4. compare zeroth-, first-, and second-order methods by information, use, and scale.

1 Start from the training problem

Lecture 3 specified a linear predictor and its mean squared error:

$$\hat{y}_i = wx_i + b, \qquad L(w,b) = \frac{1}{n}\sum_{i=1}^{n}\big(y_i - (wx_i + b)\big)^2. \tag{1}$$

Training finds values of w and b that make this objective small. Trying a few values by hand can build intuition. A learning procedure needs a systematic way to update them.

The model defines the predictions available for a given input. The loss scores how well those predictions match the data. Optimization searches for parameter values that reduce that score.

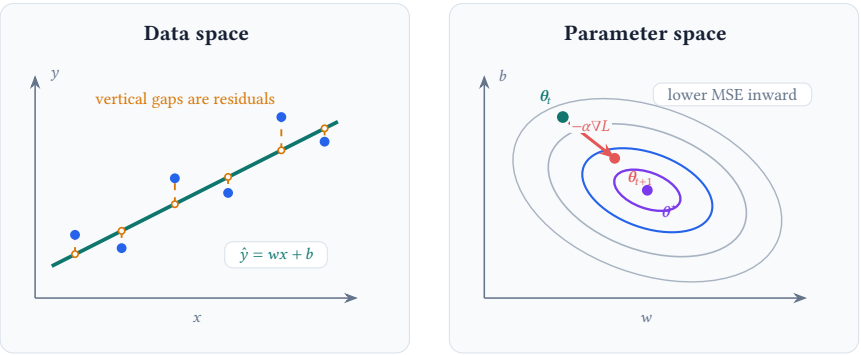

Figure 1. The figure shows the current fit and one gradient-descent update in data space and parameter space.

2 Specify the optimization problem

In general, an optimization problem takes the form

$$\theta^\star \in \arg\min_{\theta} \; f(\theta) \qquad \text{subject to} \qquad g_j(\theta) \le 0, \quad j = 1, \dots, m. \tag{2}$$

For one-feature linear regression, $\theta = (w, b)^\top$ and the objective f is the MSE in Equation (1). Many introductory problems have no explicit constraints.

Part	Linear regression	Question it answers
Variables	$\theta = (w, b)^\top$	Which values may the optimizer change?
Objective	$L(w, b)$	How does the system score a candidate model?
Constraints	feasible values of θ	Which candidate models are allowed?

Table 1. An optimization problem is defined by its variables, objective, and any constraints.

Specifying an optimization problem creates a contract with the ML model: the variables define what may change, the constraints define what is allowed, and the loss or reward defines what counts as success. This parallels contract design in economics, where incentives are chosen to shape an agent's behavior when every action cannot be prescribed directly. Designing losses and rewards remains an active area of research because a learned system follows the objective we provide, including any gaps between that objective and the outcome we intended.

The shape of the objective determines what a search method can guarantee. A *global minimum* has the lowest objective value over the full feasible set. At a *local minimum*, no nearby point has a lower value.

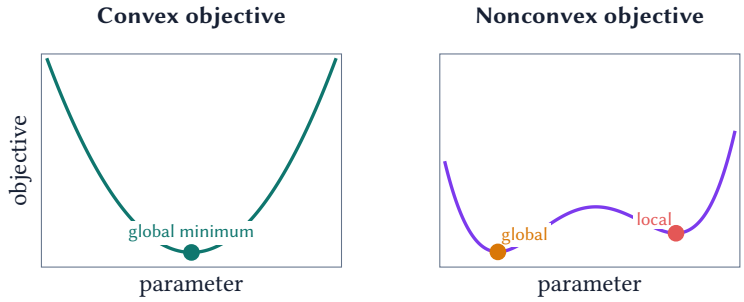

Figure 2. For a convex objective, every local minimum is global. A nonconvex objective can have several valleys with different depths. Linear regression with MSE has a convex quadratic objective in its parameters.

3 Gradient descent turns loss into updates

For the model and loss in Equation (1), differentiating the MSE gives

$$\frac{\partial L}{\partial w} = -\frac{2}{n} \sum_{i=1}^{n} x_i \big(y_i - \hat{y}_i\big), \tag{3}$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum_{i=1}^{n} \big(y_i - \hat{y}_i\big). \tag{4}$$

Near the current parameters, the gradient $\nabla L(\theta)$ points in the direction in which the loss increases fastest. Gradient descent takes a step in the

opposite direction:

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t), \tag{5}$$

where $\alpha > 0$ is the learning rate. It controls how far the parameters move at each update.

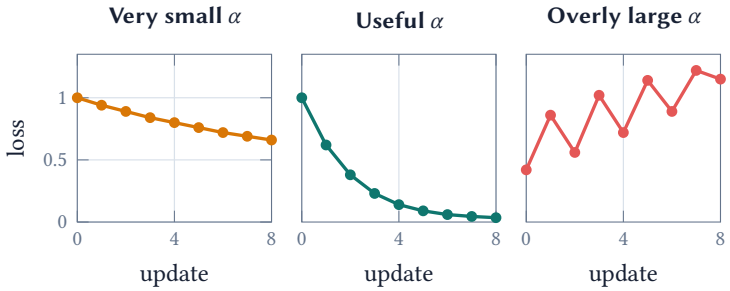

Figure 3. The learning rate sets the size of each update. A very small value makes slow progress. A useful value reduces the loss steadily. An overly large value can overshoot and diverge. The curves are conceptual.

The same update can use different amounts of data to estimate the gradient:

Method	Examples per update	Practical behavior
Batch gradient descent	all n examples	Exact gradient of the empirical loss; each update can be expensive.
Stochastic gradient descent	one sampled example	Inexpensive, frequent, and noisy updates.
Mini-batch gradient descent	a small sampled group	Balances computation with a less noisy gradient estimate; common in practice.

Table 2. Examples per update change the computation and variability of the gradient estimate.

When examples are sampled appropriately, all three methods target the same training objective. They differ in the cost and variability of each update. A fair comparison should track both passes through the data and the total number of parameter updates.

4 Optimization methods by order

The *order* of an optimization method describes the derivative information available to it. Additional information can improve the search direction; it also raises the cost of computation and storage.

Order	Information used	Representative methods	Applications and tradeoff
Zeroth	values of $f(\boldsymbol{\theta})$	finite differences, random directions, SPSA	Simulators, physical experiments, and black-box systems; function queries and dimension can make estimates costly or noisy.
First	gradients or stochastic gradients	GD, SGD, momentum, Nesterov acceleration, AdaGrad, Adam	Large datasets and high-dimensional models; inexpensive vector updates often require many iterations.
Second	gradients and curvature	Newton, Gauss–Newton, Levenberg–Marquardt	Smooth, moderate-size problems; strong directions and fast local convergence come with expensive curvature calculations.

Table 3. The useful method depends on which information can be obtained at a reasonable cost.

Zeroth-order methods observe only objective values, which makes them useful for black-box systems, simulators, and experiments where gradients are unavailable. A random-direction method can use one sampled perturbation to estimate first-order information in expectation for a smoothed objective, though individual estimates can be noisy. First-order methods use gradients and include SGD, momentum, Nesterov acceleration, adaptive methods such as AdaGrad and Adam, and proximal methods. Their modest cost per update makes them the standard choice for large-scale ML. Second-order methods such as Newton's method also use curvature and can converge in far fewer iterations near a solution. Computing and storing curvature may be too expensive for large datasets and models, so quasi-Newton and Hessian-free methods provide practical compromises.

Lecture summary

An optimization problem specifies the parameters, objective, and constraints; function values, gradients, and curvature determine which algorithms are practical and how per-step cost trades against convergence speed.

Acknowledgment. These notes are based on the instructor's handwritten notes and transcripts of lecture discussions and were editorially polished and typeset with assistance from OpenAI Codex and Anthropic Claude. The instructor reviewed and is responsible for the final content.